Functional (Style)

Quentin Crain

```
nums = [4, 2, 7, 22]
```

New list where each number in `nums` except 7, which is thrown out, is incremented by 3.

Before

```
nums = [4, 2, 7, 22]
```

After

```
def add(a, b):
    return a + b

nums = [4, 2, 7, 22]

new_nums = []

for num in nums:
    if num != 7:
        new_nums.append(add(3, num))

return new_nums
```

`3,` is added cognitive load in `add(3, num)`, lets get rid of it!

Before

```
def add(a, b):
    return a + b

nums = [4, 2, 7, 22]

new_nums = []

for num in nums:
    if num != 7:
        new_nums.append(add(3, num))

return new_nums
```

After

```
import functools

def add(a, b):
    return a + b
add_3 = functools.partial(add, 3)

nums = [4, 2, 7, 22]

new_nums = []

for num in nums:
    if num != 7:
        new_nums.append(add_3(num))

return new_nums
```

Simple `for` loops should be list comprehensions.

Before

```
import functools

def add(a, b):
    return a + b
add_3 = functools.partial(add, 3)

nums = [4, 2, 7, 22]

new_nums = []

for num in nums:
    if num != 7:
        new_nums.append(add_3(num))

return new_nums
```

After

```
import functools

def add(a, b):
    return a + b
add_3 = functools.partial(add, 3)

nums = [4, 2, 7, 22]

!new_nums = [add_3(num) for num in nums if num != 7]

return new_nums
```

Make condition a function.

Before

```
import functools

def add(a, b):
    return a + b
add_3 = functools.partial(add, 3)

nums = [1, 2, 7, 22]

new_nums = [add_3(num) for num in nums if num != 7]

return new_nums
```

After

```
import functools
!import operator

def add(a, b):
    return a + b
add_3 = functools.partial(add, 3)

!ne_7 = functools.partial(operator.ne, 7)

nums = [1, 2, 7, 22]

!new_nums = [add_3(num) for num in nums if ne_7(num)]

return new_nums
```

The list comprehension is a `filter` then a `map`.

Before

```
import functools
import operator

def add(a, b):
    return a + b
add_3 = functools.partial(add, 3)

ne_7 = functools.partial(operator.ne, 7)

nums = [4, 2, 7, 22]

new_nums = [add_3(num) for num in nums if ne_7(num)]

return new_nums
```

After

```
import functools
import operator

def add(a, b):
    return a + b
add_3 = functools.partial(add, 3)

ne_7 = functools.partial(operator.ne, 7)

nums = [4, 2, 7, 22]

!nums_without_7 = filter(ne_7, nums)

!new_nums = map(add_3, nums_without_7)

return new_nums
```

Explicit looping is gone, loops are "higher-order" concepts of `map`, `filter` (and `reduce`).

Before

```
def add(a, b):
    return a + b

nums = [4, 2, 7, 22]

new_nums = []

for num in nums:
    if num != 7:
        new_nums.append(add(3, num))

return new_nums
```

After

```
import functools
import operator

def add(a, b):
    return a + b
add_3 = functools.partial(add, 3)

ne_7 = functools.partial(operator.ne, 7)

nums = [4, 2, 7, 22]

nums_without_7 = filter(ne_7, nums)

new_nums = map(add_3, nums_without_7)

return new_nums
```

COMMENTS

QUESTIONS

CONCERNS

Functional is better!

Elixir

```
iex> nums = [4, 2, 7, 22]
[4, 2, 7, 22]

iex> nums
     |> Enum.filter(fn n -> n != 7 end)
     |> Enum.map(fn n -> n + 3 end)
[7, 5, 25]

# or, with the capture shorthand:
iex> nums
     |> Enum.filter(& &1 != 7)
     |> Enum.map(& &1 + 3)
[7, 5, 25]
```

Racket

```
> (let [(nums '(4 2 7 22))
        (add-3 (curry + 3))
        (ne-7 (compose not (curry = 7)))]
    (map add-3 (filter ne-7 nums))
    )
'(7 5 25)

; absurd, or is it?!?
(let* [(nums '(4 2 7 22))
       (add-3 (curry + 3))
       (map-add-3 (curry map add-3))
       (ne-7 (compose not (curry = 7)))
       (filter-ne-7 (curry filter ne-7))
       (filter-ne-7-then-map-add-3 (compose map-add-3 filter-ne-7))]
    (filter-ne-7-then-map-add-3 nums)
    )
```